

Controller Calibration Guide

E-puck Vision Navigation — Floor Plane Segmentation Controller

Table of Contents

1. [Overview and Calibration Philosophy](#)
 2. [Parameter Reference Map](#)
 3. [Phase 1 — Floor Model Calibration](#)
 4. [Phase 2 — Obstacle Detection Calibration](#)
 5. [Phase 3 — Navigation Behaviour Calibration](#)
 6. [Phase 4 — Avoidance Manoeuvre Calibration](#)
 7. [Validation Procedures](#)
 8. [Common Issues and Fixes](#)
 9. [Quick-Reference Tuning Table](#)
-

1. Overview and Calibration Philosophy

The controller has **two independent subsystems** that must be calibrated separately and in order. Calibrating them out of order introduces confounding errors — an unstable floor model will produce bad obstacle masks, which will make avoidance timing appear wrong even when it is correctly set.

Calibration Order:

- ① Floor Model (perception foundation)
↓
- ② Obstacle Detection Sensitivity
↓
- ③ Navigation Heading and Speed
↓
- ④ Avoidance Manoeuvre Timing

Golden rule: Only change one parameter at a time. Run at least 3 full goal-navigation trials per change before concluding anything.

Debug windows are your primary calibration instrument. Keep `SHOW_DEBUG_WINDOWS = True` throughout all calibration sessions. The three windows to watch:

Window	What it shows	Used for
Robot Camera View	Live frame + red obstacle overlay + green floor sample box	Checking detection quality visually
Floor Model Mask	Binary obstacle mask from Mahalanobis distance	Tuning MAHAL_THRESHOLD
Legacy Mask (gray threshold)	Old brightness-threshold mask	Side-by-side comparison

2. Parameter Reference Map

All tunable parameters are grouped by their subsystem below. Numbers in brackets show recommended starting values and safe adjustment ranges.

2.1 Simulation Timing

Parameter	Default	Range	Effect
<code>TIME_STEP</code>	32	32-64	Must equal <code>basicTimeStep</code> in .wbt file
<code>WARMUP_STEPS</code>	3	2-10	Steps before control starts; increase if GPS/compass read NaN on startup

2.2 Goal Navigation

Parameter	Default	Range	Effect
<code>GOAL_X</code>	2.5	—	Goal X coordinate in metres
<code>GOAL_Y</code>	-0.5	—	Goal Y coordinate in metres
<code>GOAL_TOLERANCE_M</code>	0.01	0.005-0.05	Arrival radius; tighten for precision, loosen for reliability
<code>FORWARD_SPEED</code>	0.75 × MAX	0.4-0.85 × MAX	Cruising speed; lower = more stable, higher = faster runs

Parameter	Default	Range	Effect
STEER_SPEED	$0.35 \times \text{MAX}$	$0.2\text{--}0.5 \times \text{MAX}$	Outer-wheel speed during proportional steering
TURN_SPEED	$0.55 \times \text{MAX}$	$0.3\text{--}0.7 \times \text{MAX}$	In-place rotation speed
HEADING_KP	1.6	0.8–3.0	Heading error proportional gain
HEADING_DEADBAND	0.08 rad	0.04–0.15	Error below which heading is "close enough"
LARGE_HEADING_ERR	0.50 rad	0.3–0.8	Threshold for switching from steer to rotate
SLOWDOWN_RADIUS	0.10 m	0.05–0.25	Distance from goal at which robot begins slowing down
MIN_SPEED_FRAC	0.25	0.1–0.5	Minimum speed fraction when near goal

2.3 Avoidance Timing

Parameter	Default	Range	Effect
TURN_DURATION	15 steps	8–30	How long the robot turns during Phase 1 avoidance
STRIDE_DURATION	45 steps	20–80	How long the robot drives forward during Phase 2 (the stride)
AVOID_COOLDOWN	12 steps	5–25	Steps after avoidance ends before new detection is permitted

2.4 Floor Model

Parameter	Default	Range	Effect
FLOOR_SAMPLE_H_FRAC	0.20	0.10–0.35	Height of floor sample strip (fraction of frame height, from bottom)
FLOOR_SAMPLE_W_FRAC	0.50	0.25–0.75	Width of floor sample strip (centred)
MAHAL_THRESHOLD	3.0	1.5–6.0	Sensitivity threshold — core tuning knob
FLOOR_LEARN_ALPHA	0.08	0.0–0.20	Online model adaptation speed

Parameter	Default	Range	Effect
<code>OBSTACLE_ZONE_H_FRAC</code>	0.30	0.15-0.50	Top fraction of frame excluded from obstacle detection
<code>BLOCKED_PIXEL_RATIO</code>	0.20	0.05-0.50	Fraction of zone pixels that must be non-floor to declare blockage
<code>MIN_COVARIANCE_EIG</code>	0.5	0.1-2.0	Covariance regularisation; increase if floor is very uniform

3. Phase 1 — Floor Model Calibration

The floor model is the foundation of the entire perception system. Everything downstream depends on it being correct.

Step 1.1 — Verify the floor sample region

Start the simulation and observe the **green rectangle** in the "Robot Camera View" window. This rectangle shows exactly which pixels are being used to build the floor model.

What it should look like: The rectangle sits cleanly on the flat floor surface with no obstacle, shadow, robot wheel edge, or wall visible inside it.

Common problems:

- Rectangle clips a wheel or shadow → reduce `FLOOR_SAMPLE_W_FRAC` from 0.50 to 0.35
- Rectangle is too small to capture enough floor variation → increase `FLOOR_SAMPLE_H_FRAC`
- Rectangle clips the bottom of a nearby wall → reduce `FLOOR_SAMPLE_H_FRAC`

Step 1.2 — Confirm the bootstrap LAB values

On simulation start, the console prints:

```
[FloorModel] Bootstrap mean_LAB=[L, a, b] patch_size=(H, W)
```

Expected values for typical Webots floors:

- White/light floor: `L ≈ 75-95, a ≈ -2 to 2, b ≈ -2 to 2`
- Grey floor: `L ≈ 40-60, a ≈ 0, b ≈ 0`
- Tiled floor: `L ≈ 50-80` with slightly higher `a` or `b` magnitude

Red flags:

- $L < 30$ → sample strip is in shadow or pointing at a dark object
- $L > 98$ → camera is over-exposed; check Webots rendering settings
- $|a| > 10$ or $|b| > 10$ → strip is sampling a coloured surface (not floor)

Step 1.3 — Test adaptive update behaviour

Drive the robot across a floor tile boundary (if your world has one) or into a differently lit area. Watch the console — the model should print updated mean values that drift slowly toward the new floor appearance.

If the model adapts too slowly (obstacle gets "accepted" as floor): lower `FLOOR_LEARN_ALPHA` toward 0.03.

If the model adapts too fast (false positives disappear then return): raise `FLOOR_LEARN_ALPHA` toward 0.15. Be aware that values above 0.15 risk learning obstacles during the `AVOID_DRIVE` stride phase — see Section 6.3.

4. Phase 2 — Obstacle Detection Calibration

Step 2.1 — Tune MAHAL_THRESHOLD

This is the single most important parameter. The procedure is:

1. Place the robot facing a clear path (no obstacles in view)
2. Check the "Floor Model Mask" window — it should be **entirely black** (all floor)
3. If white pixels appear on the clear floor: **raise** `MAHAL_THRESHOLD` by 0.5 until the false positives disappear
4. Now place the robot facing each obstacle type in turn
5. The mask should show a clear **white region** covering the obstacle's lower portion
6. If an obstacle produces no white pixels: **lower** `MAHAL_THRESHOLD` by 0.5

Typical settled values by floor type:

- Uniform white/grey floor: 2.5-3.5
- Mildly textured floor: 3.5-4.5
- Highly textured or patterned floor: 4.5-5.5

Step 2.2 — Tune BLOCKED_PIXEL_RATIO

This controls how many obstacle pixels must appear in a zone before it is declared "blocked". It acts as a noise filter.

Testing procedure:

1. Drive toward a thin obstacle (like `obstacle_5_orange_wall` in the test world)
2. Watch the console for `[OBS]` lines — they should appear when the obstacle occupies the central zone
3. If detection triggers too late (robot is nearly touching): **lower** to 0.10–0.15
4. If detection triggers on shadows or floor variations: **raise** to 0.25–0.35

Step 2.3 — Tune OBSTACLE_ZONE_H_FRAC

This masks out the top portion of the frame to prevent distant walls and ceiling from triggering false detections.

1. Drive the robot to a corner of the arena and face a wall from 1 metre away
 2. The "Floor Model Mask" window should show white pixels only in the lower portion, not the top
 3. If distant walls cause white regions in the top strip: raise `OBSTACLE_ZONE_H_FRAC` to 0.40
 4. If close obstacles are partially cut off at the top: lower to 0.20
-

5. Phase 3 — Navigation Behaviour Calibration

Step 3.1 — HEADING_KP (proportional gain)

Symptom: Robot oscillates/weaves while driving forward Cause: `HEADING_KP` is too high. The correction overshoots and causes left-right oscillation. Fix: Reduce from 1.6 toward 1.0–1.2 in steps of 0.2.

Symptom: Robot takes very wide curves to align with the goal Cause: `HEADING_KP` is too low. Fix: Raise from 1.6 toward 2.0–2.5 in steps of 0.2.

Step 3.2 — LARGE_HEADING_ERR

This threshold switches the robot from proportional steering into an in-place rotate. Setting it correctly determines whether the robot gracefully curves toward the goal or sharply pivots.

- If robot over-rotates and misses the goal angle: lower to 0.35–0.40 rad (~20–23°)

- If robot attempts to steer with a very large heading error and drifts wide: raise to 0.60–0.70 rad ($\sim 34\text{--}40^\circ$)

Step 3.3 — FORWARD_SPEED and STEER_SPEED

The difference $(\text{FORWARD_SPEED} - \text{STEER_SPEED})$ sets the maximum lateral correction authority. If this gap is too small, the robot cannot steer sharply enough; too large, it weaves.

Recommended ratio: $\text{STEER_SPEED} \approx 0.4\text{--}0.5 \times \text{FORWARD_SPEED}$

Step 3.4 — SLOWDOWN_RADIUS and MIN_SPEED_FRAC

If the robot overshoots the goal marker and oscillates around it:

- Increase (SLOWDOWN_RADIUS) from 0.10 to 0.20–0.30 m
- Decrease (MIN_SPEED_FRAC) from 0.25 to 0.10–0.15

If the robot stops too early (far from goal):

- Decrease (SLOWDOWN_RADIUS)
- Tighten $(\text{GOAL_TOLERANCE_M})$ to 0.005–0.008 m

6. Phase 4 — Avoidance Manoeuvre Calibration

Step 6.1 — TURN_DURATION

This controls how far the robot rotates sideways during Phase 1 (the turn). At $(\text{TURN_SPEED} = 0.55 \times 6.28 = 3.45 \text{ rad/s})$, with $(\text{TIME_STEP} = 0.032 \text{ s})$:

$$\text{Rotation per step} \approx \text{TURN_SPEED} \times \text{TIME_STEP} / \text{WHEEL_BASE_RADIUS}$$

For the e-puck (wheel base $\approx 52 \text{ mm}$, wheel radius $\approx 20 \text{ mm}$):

- 15 steps $\approx 35\text{--}40^\circ$ of rotation

If robot barely clears obstacles: increase (TURN_DURATION) to 20–25 steps. **If robot turns so far it misses the goal and circles:** reduce to 10–12 steps.

Step 6.2 — STRIDE_DURATION

This controls how far the robot travels sideways during Phase 2 (the forward stride). At

$(\text{FORWARD_SPEED} = 0.75 \times 6.28 = 4.71 \text{ rad/s})$ motor speed:

- $45 \text{ steps} \times 0.032 \text{ s} \times \sim 0.12 \text{ m/s} \approx 0.17 \text{ m}$ of lateral travel

If robot does not fully clear obstacles before re-entering SEEK: increase to 55-70 steps. If robot overshoots the goal laterally: reduce to 25-35 steps.

Step 6.3 — AVOID_COOLDOWN and FLOOR_LEARN_ALPHA interaction

The cooldown prevents re-triggering immediately after an avoidance. However, during `AVOID_DRIVE`, the floor model still updates via EMA — if `FLOOR_LEARN_ALPHA` is too high and the robot is driving past an obstacle, the obstacle's colour begins bleeding into the floor model.

Safe rule: `AVOID_COOLDOWN` steps $\times$ `TIME_STEP` should be at least $2-3\times$ the floor model's time constant ($(1 / \text{FLOOR_LEARN_ALPHA}) \text{ steps} \approx 12 \text{ steps at } \alpha=0.08$). The default cooldown of 12 steps satisfies this. If you raise `FLOOR_LEARN_ALPHA`, raise `AVOID_COOLDOWN` proportionally.

7. Validation Procedures

Test 7.1 — Straight-Line Baseline

Setup: Remove all obstacles. Place robot at (0,0) facing goal at (2.5, -0.5). **Pass criteria:**

- Robot reaches goal within 500 steps
- No false-positive obstacle detections (console shows no `[OBS]` lines)
- "Floor Model Mask" stays black throughout

Fail actions:

- False positives $\rightarrow$ raise `MAHAL_THRESHOLD`
- Robot does not reach goal $\rightarrow$ check `GOAL_TOLERANCE_M` and GPS values

Test 7.2 — Single White Obstacle

Setup: Place only `obstacle_2_white_cylinder` (white) directly in the robot's path at (0.8, 0).

Pass criteria:

- `[OBS]` line appears in console when obstacle enters the center zone
- Robot avoids without collision
- Robot resumes goal-seeking after avoidance

Fail actions:

- Obstacle not detected $\rightarrow$ lower `MAHAL_THRESHOLD` by 0.5, retest
- False trigger before obstacle is visible $\rightarrow$ raise `MAHAL_THRESHOLD` by 0.5

Test 7.3 — Stride Blind-Spot (the core issue)

Setup: Place a second obstacle 0.3 m ahead of the first, offset 0.1 m. **Pass criteria:**

- Robot detects the second obstacle during the stride phase and aborts the stride
- No collision occurs

Test 7.4 — Full Course

Setup: Run the complete world with all 5 diverse obstacles. **Pass criteria:**

- Robot reaches goal (2.5, -0.5) without collision in at least 4 out of 5 runs
 - Console shows obstacle detection events for at least 4 of the 5 obstacles
-

8. Common Issues and Fixes

Issue: Robot spins in place indefinitely

Cause: `avoid_direction` is being set to 0 (neutral) somehow, or the turn phase loops. **Fix:** Print `avoid_direction` at avoidance entry. Ensure `lm` and `rm` are not both exactly equal. Add a fallback: if `lm == rm`, default to `avoid_direction = 1`.

Issue: Floor model "learns" obstacles and stops detecting them

Cause: `FLOOR_LEARN_ALPHA` too high combined with slow approach speed — the obstacle's colour gradually enters the running mean. **Fix 1:** Lower `FLOOR_LEARN_ALPHA` to 0.03–0.05. **Fix 2:** Freeze the floor model during `AVOID_DRIVE` state (call `update()` only in `STATE_SEEK`). The improved controller implements this fix.

Issue: Constant false detections on floor shadow or edge

Cause: Robot body shadow or wheel edges fall within the floor sample strip. **Fix:** Reduce `FLOOR_SAMPLE_W_FRAC` from 0.50 to 0.30 to narrow the sample away from the robot's lateral edges.

Issue: Thin obstacles not detected until collision

Cause: `BLOCKED_PIXEL_RATIO = 0.20` is too high for thin targets. A 4 cm wide obstacle at 0.5 m occupies only 5–8% of zone width. **Fix:** Lower `BLOCKED_PIXEL_RATIO` to 0.08–0.12 for environments with thin obstacles.

Issue: Robot overshoots goal and circles

Cause: `SLOWDOWN_RADIUS` is too small — robot is still at full speed when it reaches arrival

tolerance. **Fix:** Raise `SLOWDOWN_RADIUS` to 0.20–0.30 m and lower `MIN_SPEED_FRAC` to 0.10.

Issue: Robot re-enters avoidance immediately after `AVOID_COOLDOWN` expires

Cause: Cooldown is too short — robot hasn't moved far enough past the obstacle, which still occupies the center zone. **Fix:** Raise `AVOID_COOLDOWN` to 20–25 steps, or raise `STRIDE_DURATION` so the robot clears the obstacle fully.

Issue: GPS returns NaN on first steps

Cause: Webots GPS needs one step to initialise. **Fix:** Raise `WARMUP_STEPS` from 3 to 5–8.

Issue: `ARRIVED` state never triggered despite robot being at goal

Cause: `GOAL_TOLERANCE_M` too tight (0.01 m) combined with GPS noise. **Fix:** Raise to 0.02–0.03 m, or use a rolling average of the last 3 GPS readings.

9. Quick-Reference Tuning Table

Symptom	Parameter to adjust	Direction
False obstacle detections on clear floor	<code>MAHAL_THRESHOLD</code>	↑ Raise
Obstacles not detected (missed)	<code>MAHAL_THRESHOLD</code>	↓ Lower
Thin obstacles missed	<code>BLOCKED_PIXEL_RATIO</code>	↓ Lower
False triggers from shadows	<code>BLOCKED_PIXEL_RATIO</code>	↑ Raise
Heading oscillation/weave	<code>HEADING_KP</code>	↓ Lower
Slow heading correction	<code>HEADING_KP</code>	↑ Raise
Robot turns too far in avoidance	<code>TURN_DURATION</code>	↓ Lower
Robot doesn't clear obstacle	<code>TURN_DURATION</code> + <code>STRIDE_DURATION</code>	↑ Raise
Floor model learns obstacles	<code>FLOOR_LEARN_ALPHA</code>	↓ Lower
Model slow to adapt to new floor	<code>FLOOR_LEARN_ALPHA</code>	↑ Raise
Distant walls trigger detection	<code>OBSTACLE_ZONE_H_FRAC</code>	↑ Raise
Close obstacles cut off at top	<code>OBSTACLE_ZONE_H_FRAC</code>	↓ Lower
Overshoot at goal	<code>SLOWDOWN_RADIUS</code>	↑ Raise

Symptom	Parameter to adjust	Direction
Stops too early	SLOWDOWN_RADIUS + GOAL_TOLERANCE_M	↓ Lower
Cooldown too short, re-triggers	AVOID_COOLDOWN	↑ Raise